

The Builder: The Dashboard Component Factory



Figure 1: Every task begins with a single brick.

RosettaDash's builder is an Angular client using a NestJS API, both local. From the repo root:

```
npm install  
npm start
```

Open <http://localhost:4200>, where the angular app runs; the NestJS app is running on <http://localhost:3000>. You need both processes, which launch together. Save, preview, and export talk to the API. This is probably the geekies article in the series.

Getting Started

You select a front end framework, which determines future choices of servers, database, and styling suggestions. Certain libraries and applications favor or are only available to specific frameworks, such as Nuxt, mui, and others. The secondary selections change when you change base framework.

After you have selected Web Components, React, Angular, Vue, or Svelte, choose the mix of server and database (if any) along with how you will style the components, then press Continue to builder.

If you have a small screen, you will immediately see an alert saying that you need a screen width of 1024 pixel or greater. Truthfully, the builder is more suited for large desktop screens, at least 1280 or more pixels.

Becoming Familiar with the Interface

The sections of the interface are: the components are grouped on the left side of the screen in a palette. Next is the canvas, where components are placed by selecting the + button on their component. Once on the canvas, their details are available in the Inspector panel on the right. At the top of the screen, choices like design and preview, undo and redo, create, how it works, AI assist and export, and save and save to library provide the ways to see and work with the components, and how to create, export and save the components that you have created.

COMPONENTS

Click to inspect, + to add, i = how it works, = what it goes with

▼ Form Inputs7

Text Input

i = +

Select

i = +

Number Input

i = +

Checkbox

i = +

Textarea

i = +

Date Range

i = +

Time Preset

i = +

> Data Display3

> Logic & Motion2

> Charts3

> Views3

> Layout & Navigation6

> Access & Onboarding3

> Data Sources5

> API Servers4

CANVAS

Figure 2: Components palette.

Components. On the left side, the components are arranged in groups. You will recognize many: a KPI (key performance indicator), a table, a date range, a grid, a chart, and many others. Clicking the components header collapses it to the left side.

Canvas. In the center of the app, you can place, snap, resize, and multi-select in the canvas. The canvas is for the positioning and sizing of the component elements for your exported component.

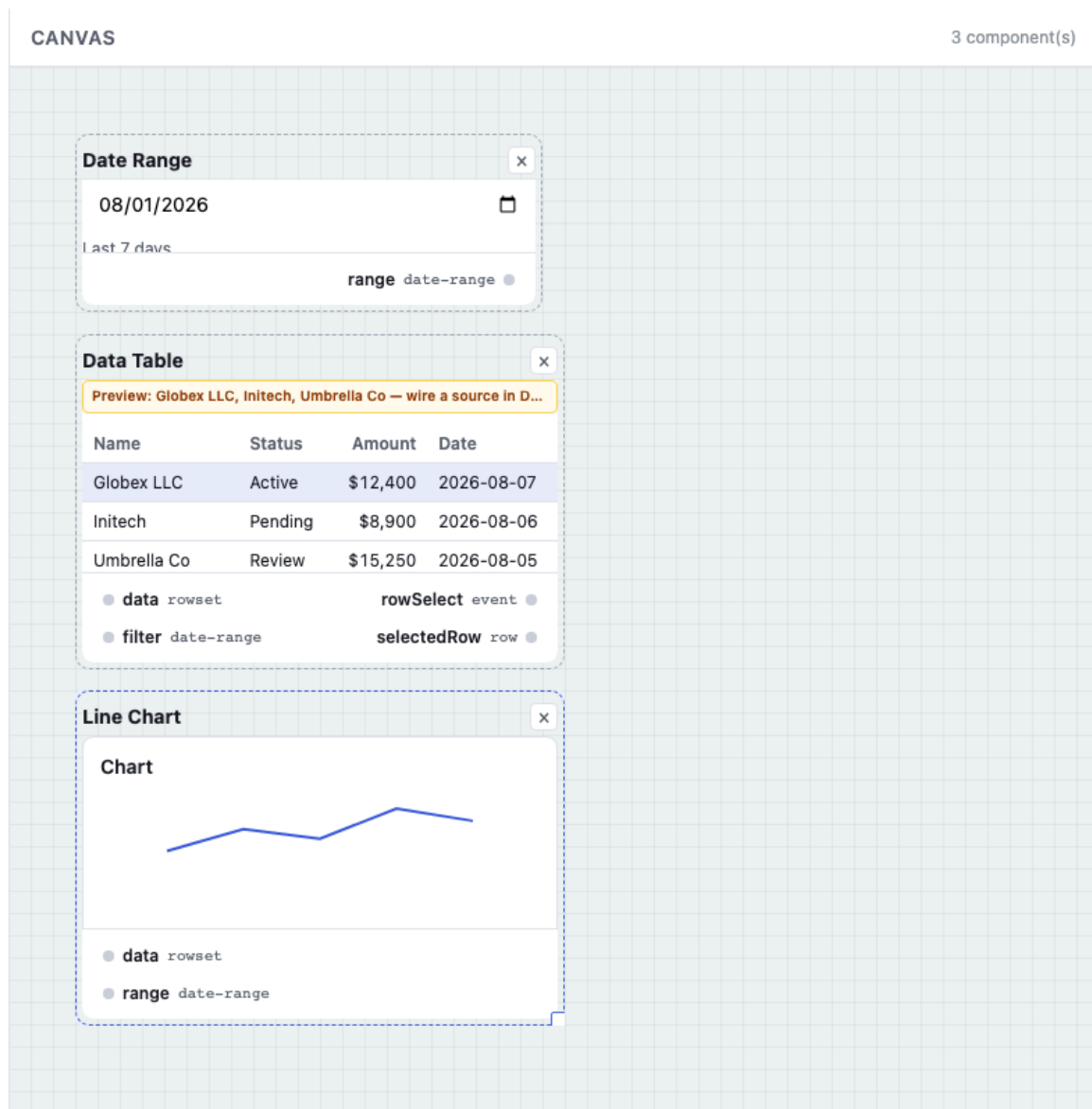


Figure 3: Canvas with a binding — date range to table to chart.

Inspector. On the right side, the Inspector is activated when something is selected on the canvas. Properties, suggestions, data sources and other details are visible here.

INSPECTOR

Upload or link an equirectangular (or flat) video file for downstream processing

▼ Placement

X

112

Y

512

W

640

H

480

▼ Suggestions

2

Typical grouping

Consider adding Flat Video Viewport. Place above equirect viewport and WASM media extract components.

Dismiss

Typical grouping

Consider adding Live Capture. Place above equirect viewport and WASM media extract components.

Dismiss

▼ Properties

6

Name

Video Source

Label

Remove from canvas

Figure 4: Inspector details update as you interact with element.

Preview.

Preview mock data comes from the builder API on :3000 — the same preview-content.json rows the canvas describes. That is not the same as proving **your** Welcome stack in Docker (see Nodes that never render below): parity boots the **database profile** and **server profile** that match what you picked on Welcome, not every engine at once.

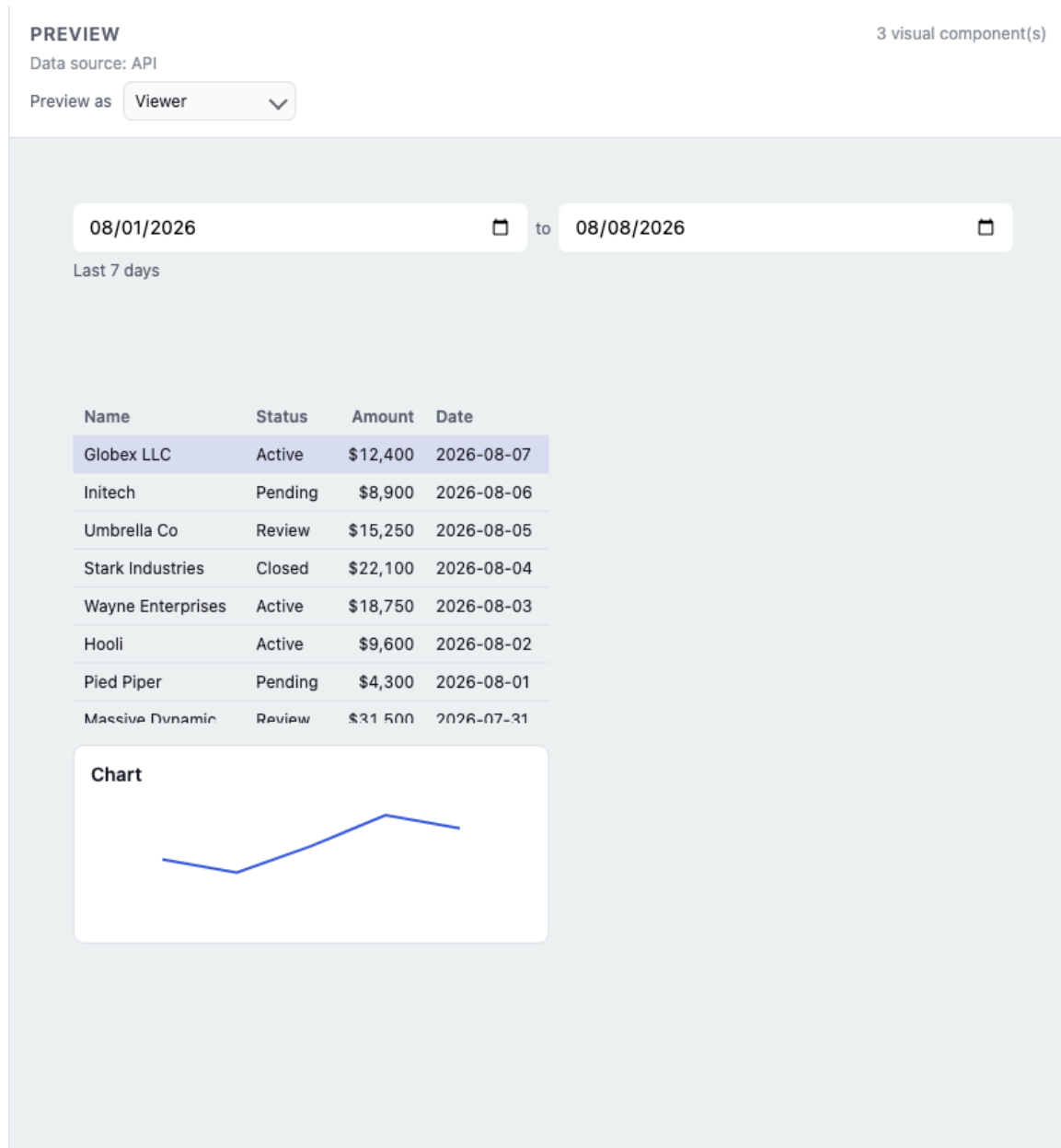


Figure 5: Preview — mock data, filter clicked, table moving.

Export. Full composite, a single node, or a selection neighborhood (the pieces you highlighted plus what they need). Download source. For a UI-only stack you get templates, styles, and scripts.

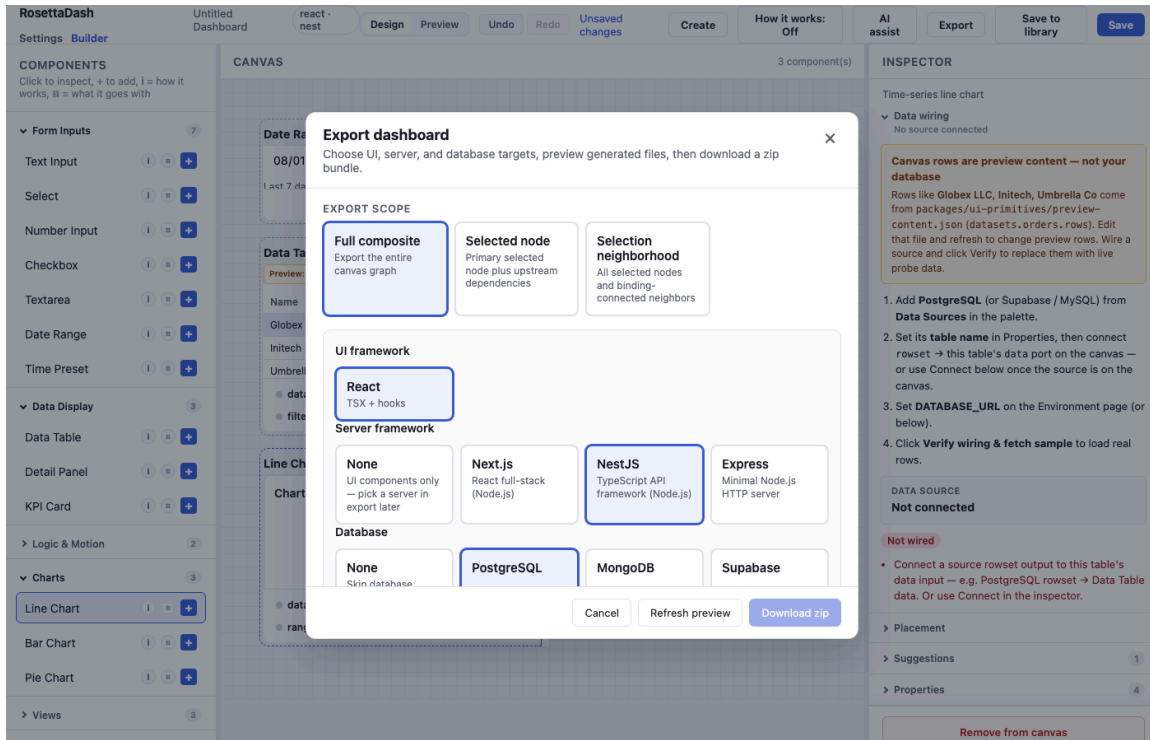


Figure 6: Export wizard — full composite, single node, or a neighborhood.

Nodes that never render

This section applies to backend for frontend (BFF) developers that connect data to the front end. You choose database and server targets the same way you chose a framework. The builder generates config, stubs, and .env templates when you export. Connection strings stay in env vars. They are not hard-coded in the zip.

Proving your Welcome stack

The repo ships a **parity stack** that compiles with real drivers and serves seeded rows from Docker. It pairs with the **database and server you chose on Welcome** (see Getting Started above).

Once per machine — `start:server` blocks its terminal; use a second shell for generate and Docker:

```
npm run start:server          # terminal A — builder API :3000
npm run parity:generate        # terminal B — after API is up
```

Match your Welcome picks — compose profiles and check flags:

Welcome UI	Server to prove	Compose profile	Check server	Port
React	Next.js	server-next	--only next	53103
Angular	NestJS	server-nest	--only nest	53101
Vue	Nuxt	server-nuxt	--only nuxt	53104
Svelte	Express	server-express	--only express	53102
Web Components	Express	server-express	--only express	53102

Welcome database	Compose profile	Check database
PostgreSQL	postgres	--only postgres
MySQL	mysql	--only mysql
MongoDB	mongo	--only mongo
Supabase	supabase	--only supabase

Example — React + PostgreSQL + Next.js (idiomatic React full stack):

```
docker compose --profile postgres --profile server-next up -d
node tools/backend-parity/check-databases.mjs --only postgres
node tools/backend-parity/check-servers.mjs --only next
curl http://127.0.0.1:53103/api/orders
```

Swap the two compose profiles and --only values for your stack. The generated app reads the same seeded orders table as builder preview. Storybook and Destination Atlas Stack tabs can fetch that live API when parity is up (articles 3 and 6). Full matrix: docs/44-backend-parity-stack.md.

Composite components

A **composite component** is a group of components that work as a unit. You can save it, version it, export it as a page or a module. The weather widget in demos/ is that idea at card scale. Destination Atlas, later in this series, is the same idea at app scale.

Next

The next article is Storybook: same pieces, isolated, before they are part of a product.